M6 – AUTOENCODER PRACTICAL

Application of an Undercomplete Autoencoder for Sensor Anomaly Detection in SafeBand AI

Student Detail	Value
Student Name	Prabhatkumar Jha
Roll No.	08
Class / Division	BE EXTC
College	Xavier Institute of Engineering
Academic Year	2026–27

1. Objective

To apply an Undercomplete Autoencoder to multisensor data from the SafeBand AI project for learning a compact latent representation, reconstructing sensor measurements, and demonstrating reconstruction-error-based anomaly detection.

2. Introduction

An Autoencoder is an encoder–decoder model that learns to represent input data in a latent space and reconstruct the original input from that representation. The encoder maps the input to a latent representation and the decoder reconstructs the input.

An Undercomplete Autoencoder uses a latent representation with fewer dimensions than the input. The bottleneck encourages the model to retain important information in a compact representation. For this practical, the Undercomplete Autoencoder is applied to SafeBand sensor data for reconstruction and anomaly detection.

3. SafeBand AI Project Application

SafeBand AI is an intelligent safety-band project that processes multiple sensor measurements as part of its sensing and safety pipeline. The Autoencoder is added as a complementary data-analysis component.

The practical question is whether the Autoencoder can learn the normal structure of SafeBand sensor measurements and use reconstruction error as an additional indicator of an unusual sensor pattern.

4. Dataset and Input Features

Dataset: datasets/synthetic/bno055/bno055_synthetic_demo.csv

Parameter	Value
Total samples	3,600
Input features	23
Training samples	2520
Validation samples	540
Test samples	540
Latent features	8

The 23 features used by the Autoencoder are:

Feature	Feature
accel_x_mps2	accel_y_mps2
accel_z_mps2	gyro_x_dps
gyro_y_dps	gyro_z_dps
mag_x_uT	mag_y_uT
mag_z_uT	euler_heading_deg
euler_roll_deg	euler_pitch_deg
quat_w	quat_x
quat_y	quat_z
linear_accel_x_mps2	linear_accel_y_mps2

Feature	Feature
linear_accel_z_mps2	gravity_x_mps2
gravity_y_mps2	gravity_z_mps2
temperature_c	

5. Methodology

1. Load the SafeBand BNO055 sensor dataset.
2. Select the 23 numerical sensor features.
3. Remove invalid numerical values if present.
4. Split the data into training, validation, and test sets.
5. Standardize the sensor features because they have different units and numerical ranges.
6. Train the Undercomplete Autoencoder using mean squared error (MSE) reconstruction loss.
7. Compress the 23-dimensional sensor vector into an 8-dimensional latent representation.
8. Reconstruct the original 23-dimensional sensor vector.
9. Calculate reconstruction error for each sample.
10. Set the anomaly threshold using the 95th percentile of validation reconstruction error.
11. Demonstrate anomaly detection using controlled deviations in selected sensor features.

6. Autoencoder Architecture

Stage	Layer	Output
Input	Sensor input	23
Encoder	Dense + ReLU	32
Encoder	Dense + ReLU	16
Bottleneck	Latent layer + ReLU	8
Decoder	Dense + ReLU	16
Decoder	Dense + ReLU	32
Output	Linear reconstruction	23

The model contains 2,879 trainable parameters. Since the latent dimension (8) is smaller than the input dimension (23), it is an Undercomplete Autoencoder.

7. Python Implementation

The implementation uses Python, TensorFlow/Keras, NumPy, Pandas, scikit-learn, and Matplotlib.

7.1 Training Program

```
"""
SafeBand AI - Autoencoder Practical
Experiment 1: Train an undercomplete Autoencoder on BNO055 sensor data.

Purpose:
    Learn normal multisensor patterns and use reconstruction error
    as an anomaly-detection signal.

Dataset:
    datasets/synthetic/bno055/bno055_synthetic_demo.csv
"""

from pathlib import Path
```

```
import json
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers


# ------------------------------
# Configuration
# ------------------------------
PROJECT_ROOT = Path(__file__).resolve().parents[2]
DATA_PATH = PROJECT_ROOT / "datasets" / "synthetic" / "bno055" / "bno055_synthetic_demo.csv"
RESULTS_DIR = Path(__file__).resolve().parent / "results"
RESULTS_DIR.mkdir(parents=True, exist_ok=True)

EPOCHS = 50
BATCH_SIZE = 32
RANDOM_STATE = 42

# BNO055 features used by the Autoencoder.
FEATURES = [
    "accel_x_mps2", "accel_y_mps2", "accel_z_mps2",
    "gyro_x_dps", "gyro_y_dps", "gyro_z_dps",
    "mag_x_uT", "mag_y_uT", "mag_z_uT",
    "euler_heading_deg", "euler_roll_deg", "euler_pitch_deg",
    "quat_w", "quat_x", "quat_y", "quat_z",
    "linear_accel_x_mps2", "linear_accel_y_mps2", "linear_accel_z_mps2",
    "gravity_x_mps2", "gravity_y_mps2", "gravity_z_mps2",
    "temperature_c",
]


def build_autoencoder(input_dim: int) -> keras.Model:
    """Create an undercomplete dense Autoencoder."""
    inputs = keras.Input(shape=(input_dim,), name="sensor_input")

    # Encoder
    x = layers.Dense(32, activation="relu", name="encoder_dense_1")(inputs)
    x = layers.Dense(16, activation="relu", name="encoder_dense_2")(x)
    latent = layers.Dense(8, activation="relu", name="latent_space")(x)

    # Decoder
    x = layers.Dense(16, activation="relu", name="decoder_dense_1")(latent)
    x = layers.Dense(32, activation="relu", name="decoder_dense_2")(x)
    outputs = layers.Dense(input_dim, activation="linear", name="reconstruction")(x)

    model = keras.Model(inputs, outputs, name="SafeBand_Undercomplete_Autoencoder")
    model.compile(
        optimizer=keras.optimizers.Adam(learning_rate=1e-3),
        loss="mse",
    )
    return model


def main():
    print("=" * 70)
    print("SafeBand AI - Autoencoder Practical")
    print("=" * 70)

    if not DATA_PATH.exists():
        raise FileNotFoundError(f"Dataset not found: {DATA_PATH}")

    df = pd.read_csv(DATA_PATH)

    print(f"\nDataset shape: {df.shape}")
    print("Activities:", df["activity_label"].value_counts().to_dict())
```

```
    # Keep only required numeric sensor features.
    X = df[FEATURES].astype(np.float32).copy()

    # Remove any invalid rows if present.
    X = X.replace([np.inf, -np.inf], np.nan).dropna()
    print(f"Usable sensor samples: {len(X)}")

    # Train/validation/test split.
    X_train, X_temp = train_test_split(
        X, test_size=0.30, random_state=RANDOM_STATE
    )
    X_val, X_test = train_test_split(
        X_temp, test_size=0.50, random_state=RANDOM_STATE
    )

    # Standardization is important because the sensors have different units/scales.
    scaler = StandardScaler()
    X_train_scaled = scaler.fit_transform(X_train)
    X_val_scaled = scaler.transform(X_val)
    X_test_scaled = scaler.transform(X_test)

    print(f"\nTrain samples: {len(X_train_scaled)}")
    print(f"Validation samples: {len(X_val_scaled)}")
    print(f"Test samples: {len(X_test_scaled)}")

    model = build_autoencoder(X_train_scaled.shape[1])
    model.summary()

    callbacks = [
        keras.callbacks.EarlyStopping(
            monitor="val_loss",
            patience=8,
            restore_best_weights=True,
        )
    ]

[Remaining source code is available in the project file.]
```

7.2 Anomaly Demonstration Program

```
"""
SafeBand AI - Autoencoder Practical
Experiment 2: Consistent anomaly-detection demonstration.

IMPORTANT:
The anomaly threshold is NOT recalculated from the 100 demonstration
samples. It is loaded from the validation-set threshold produced during
training. This makes the experiment methodologically consistent.
"""

from pathlib import Path
import json
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from tensorflow import keras


PROJECT_ROOT = Path(__file__).resolve().parents[2]
DATA_PATH = PROJECT_ROOT / "datasets" / "synthetic" / "bno055" / "bno055_synthetic_demo.csv"
RESULTS_DIR = Path(__file__).resolve().parent / "results"

MODEL_PATH = RESULTS_DIR / "safeband_autoencoder.keras"
SCALER_PATH = RESULTS_DIR / "scaler.npz"
METRICS_PATH = RESULTS_DIR / "metrics.json"

FEATURES = [
    "accel_x_mps2", "accel_y_mps2", "accel_z_mps2",
    "gyro_x_dps", "gyro_y_dps", "gyro_z_dps",
```

```
    "mag_x_uT", "mag_y_uT", "mag_z_uT",
    "euler_heading_deg", "euler_roll_deg", "euler_pitch_deg",
    "quat_w", "quat_x", "quat_y", "quat_z",
    "linear_accel_x_mps2", "linear_accel_y_mps2", "linear_accel_z_mps2",
    "gravity_x_mps2", "gravity_y_mps2", "gravity_z_mps2",
    "temperature_c",
]


def main():
    print("=" * 70)
    print("SafeBand AI - Consistent Autoencoder Anomaly Demonstration")
    print("=" * 70)

    for required in [MODEL_PATH, SCALER_PATH, METRICS_PATH, DATA_PATH]:
        if not required.exists():
            raise FileNotFoundError(f"Required file not found: {required}")

    model = keras.models.load_model(MODEL_PATH)

    scaler_data = np.load(SCALER_PATH, allow_pickle=True)
    mean = scaler_data["mean"]
    scale = scaler_data["scale"]

    with open(METRICS_PATH, "r", encoding="utf-8") as f:
        metrics = json.load(f)

    # Use the SAME threshold determined during training.
    threshold = float(metrics["anomaly_threshold_95_percentile"])

    df = pd.read_csv(DATA_PATH)
    X = df[FEATURES].astype(np.float32).dropna().copy()

    # Reproducible set of real SafeBand sensor samples.
    normal = X.sample(n=min(100, len(X)), random_state=42).to_numpy()

    # Create a controlled abnormal pattern in standardized space.
    # This avoids unrealistic raw-unit multiplication and makes the
    # demonstration easier to explain academically.
    normal_scaled = (normal - mean) / scale
    anomalous_scaled = normal_scaled.copy()

    # Inject a strong but controlled multisensor deviation:
    # acceleration, gyroscope and temperature become unusual.
    anomalous_scaled[:, 0:3] += 3.0
    anomalous_scaled[:, 3:6] += 3.0
    anomalous_scaled[:, -1] += 3.0

    # Reconstruction.
    normal_recon = model.predict(normal_scaled, verbose=0)
    anomaly_recon = model.predict(anomalous_scaled, verbose=0)

    normal_error = np.mean(
        np.square(normal_scaled - normal_recon), axis=1
    )
    anomaly_error = np.mean(
        np.square(anomalous_scaled - anomaly_recon), axis=1
    )

    normal_detected = normal_error > threshold
    anomaly_detected = anomaly_error > threshold

    print(f"\nTraining-derived 95% threshold : {threshold:.6f}")
    print(f"Normal mean reconstruction error: {normal_error.mean():.6f}")
    print(f"Anomaly mean reconstruction error: {anomaly_error.mean():.6f}")

    print("\nDetection summary")
    print("-" * 40)
    print(
        f"Normal samples flagged: "
        f"{normal_detected.sum()}/{len(normal_detected)}"
```

```
    )
    print(
        f"Injected anomalous samples detected: "
        f"{anomaly_detected.sum()}/{len(anomaly_detected)}"
    )

    results = pd.DataFrame({
        "normal_reconstruction_error": normal_error,
        "anomalous_reconstruction_error": anomaly_error,
        "normal_flag": normal_detected,

[Remaining source code is available in the project file.]
```

8. Training Results

Metric	Result
Epochs completed	50
Final training loss	0.018596
Final validation loss	0.017463
Mean test reconstruction error	0.017082
95% validation anomaly threshold	0.052642

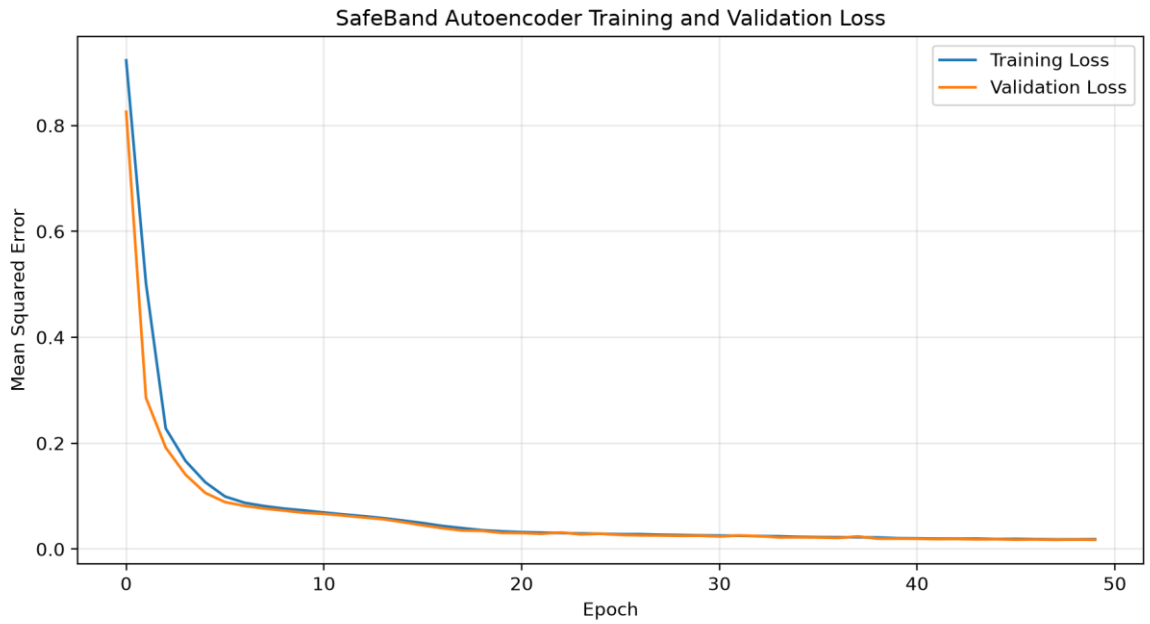


Figure 1. Training and validation reconstruction loss.

The training and validation losses decrease substantially during training and remain close toward the end of the experiment, showing that the model learned the reconstruction task.

9. Reconstruction Analysis

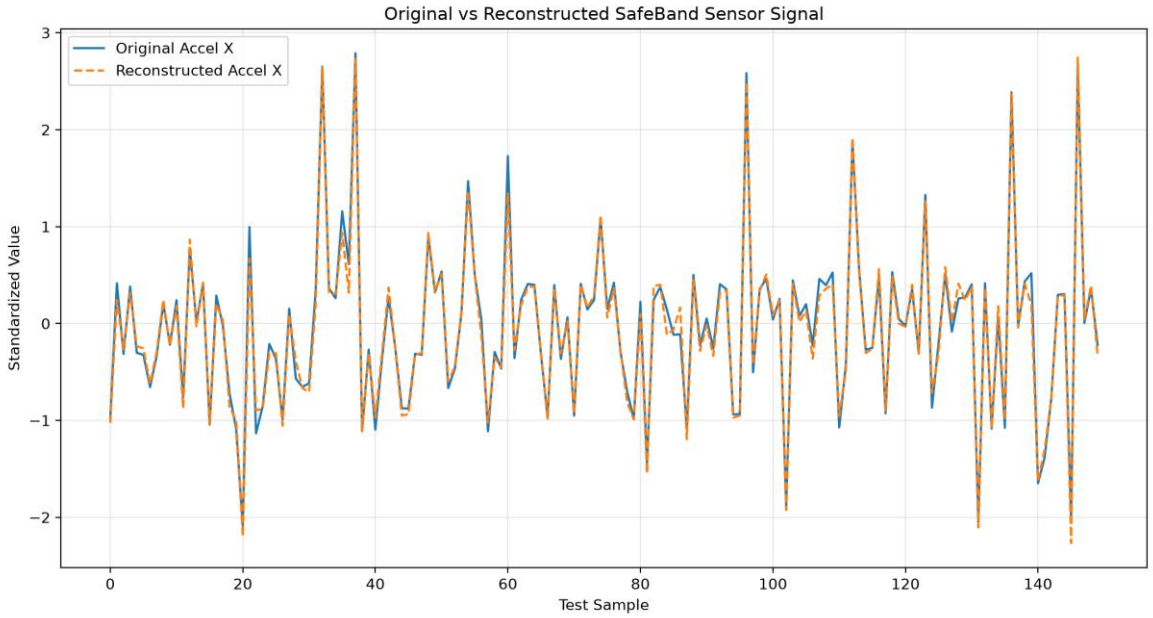


Figure 2. Original and reconstructed standardized accelerometer-X signal.

The original and reconstructed signal follow a similar pattern. The decoder reconstructs the important structure of the sensor input from the compact latent representation.

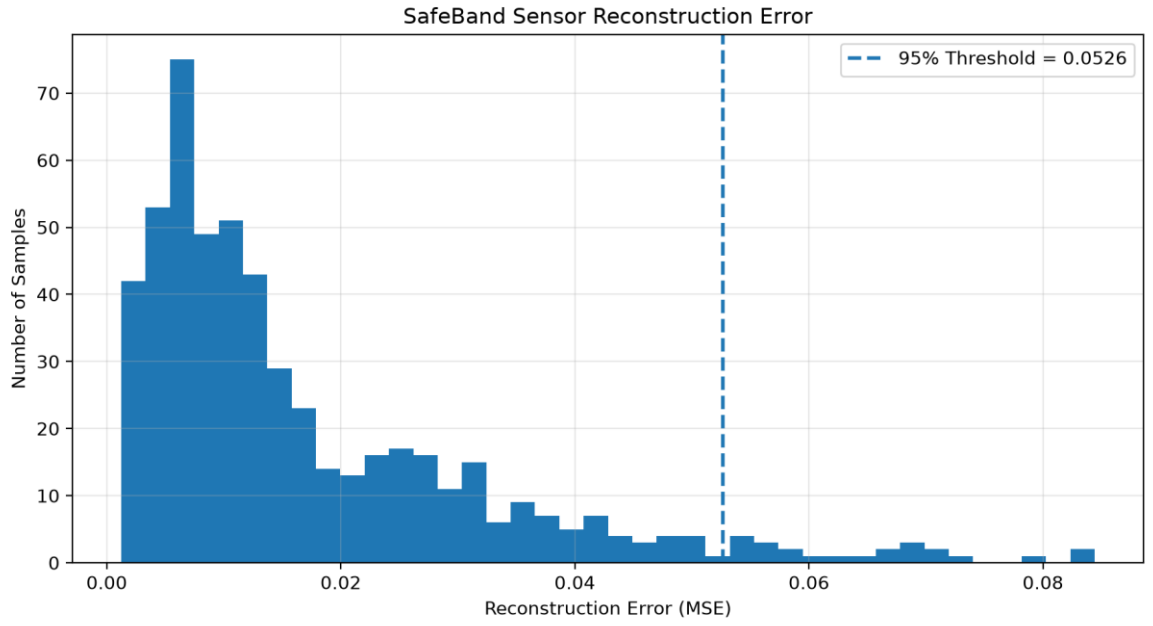


Figure 3. Test-set reconstruction-error distribution and 95% threshold.

The mean test reconstruction error is 0.017082. The threshold from the 95th percentile of validation reconstruction error is 0.052642.

10. Anomaly Detection Demonstration

Measure	Result
Training-derived threshold	0.052642
Normal mean reconstruction error	0.015626
Injected anomaly mean reconstruction error	2.123772
Normal samples flagged	3/100
Injected anomalies detected	100/100

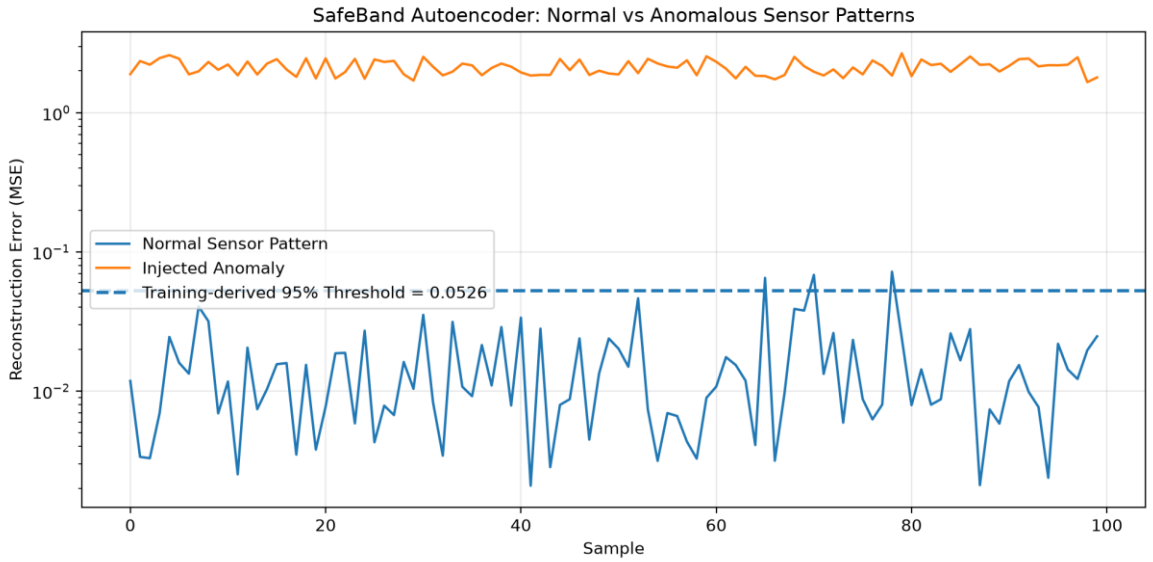


Figure 4. Normal versus controlled anomalous sensor patterns.

For the demonstration, 100 real SafeBand sensor samples were used as the normal reference set. Controlled deviations were introduced into selected standardized accelerometer, gyroscope, and temperature features. The same validation-derived threshold was used for both groups.

The normal demonstration samples had a mean reconstruction error of 0.015626, while the controlled anomalous patterns had a mean reconstruction error of 2.123772. 100 of 100 injected anomalous samples crossed the threshold, while 3 normal samples crossed it.

This is a controlled anomaly demonstration. It does not claim clinical diagnosis or clinical fall-detection accuracy.

11. Practical Application in SafeBand AI

Step	Operation
1	Collect multisensor readings.
2	Preprocess and standardize the sensor vector.
3	Pass the vector through the trained Autoencoder.
4	Reconstruct the sensor vector.
5	Calculate reconstruction error.
6	Compare the error with the validation-derived threshold.
7	Generate an anomaly signal when the error is unusually high, for further processing.

The anomaly signal can be combined with the existing SafeBand sensor-fusion and risk-processing logic. The Autoencoder therefore acts as a complementary unsupervised feature-learning and anomaly-detection component.

12. Results and Discussion

- Data representation: 23 sensor measurements are compressed into an 8-dimensional latent representation.
- Data reconstruction: the decoder reconstructs the 23-dimensional sensor vector from the latent representation.
- Feature learning: the bottleneck forces the model to retain useful information in a compact representation.
- Anomaly detection: reconstruction error provides a numerical signal for identifying unusual sensor patterns.

The decreasing training and validation losses demonstrate learning of the reconstruction task. The reconstruction plot shows that the learned representation retains the major structure of the selected sensor signal. The controlled anomaly experiment demonstrates the intended reconstruction-error mechanism.

13. Limitations

- The BNO055 dataset used for this practical is a synthetic SafeBand demonstration dataset.
- The anomalous examples were deliberately injected and are not a clinical or field-validated anomaly dataset.
- The experiment evaluates reconstruction-error separation rather than clinical diagnostic accuracy.
- The threshold is specific to the dataset, preprocessing procedure, and trained model.
- Further validation with real-world SafeBand recordings would be required before deployment.

14. Conclusion

An Undercomplete Autoencoder was successfully implemented and applied to SafeBand AI multisensor data. The model compressed 23 sensor features into an 8-dimensional latent representation and reconstructed the sensor vector through a decoder.

The experiment demonstrated decreasing reconstruction loss, similar original and reconstructed sensor patterns, and a reconstruction-error-based approach for identifying controlled anomalous sensor patterns. The practical application shows how an Autoencoder can be added to SafeBand AI as a complementary anomaly-detection mechanism.

15. Generated Outputs

- training_loss.png – training and validation loss
- original_vs_reconstructed.png – original versus reconstructed sensor signal
- reconstruction_error.png – reconstruction-error distribution and threshold
- normal_vs_anomaly_fixed.png – normal versus controlled anomalous sensor patterns
- safeband_autoencoder.keras – trained Autoencoder model

- metrics.json – recorded experiment metrics
- anomaly_detection_results_fixed.csv – anomaly demonstration results

Appendix A – Core Autoencoder Model

The following is the core model definition used in the practical.

```
def build_autoencoder(input_dim):
    inputs = keras.Input(shape=(input_dim,), name="sensor_input")

    # Encoder
    x = layers.Dense(32, activation="relu")(inputs)
    x = layers.Dense(16, activation="relu")(x)
    latent = layers.Dense(8, activation="relu",
                          name="latent_space")(x)

    # Decoder
    x = layers.Dense(16, activation="relu")(latent)
    x = layers.Dense(32, activation="relu")(x)
    outputs = layers.Dense(input_dim, activation="linear",
                           name="reconstruction")(x)

    model = keras.Model(inputs, outputs,
                        name="SafeBand_Undercomplete_Autoencoder")

    model.compile(optimizer=keras.optimizers.Adam(learning_rate=1e-3),
                  loss="mse")
    return model
```

End of report.